

Ledger-Free Trustless Battleship:

Self-Contained Zero-Knowledge Shot Proofs with Public-History Binding

Hadi Ghazi

hadigghazi@gmail.com

July 2026

Abstract

Zero-knowledge proofs let the players of an imperfect-information game answer questions about hidden state, such as whether a shot in Battleship is a hit or a miss, without revealing the state and without the ability to answer dishonestly. The zero-knowledge Battleship implementations we are aware of all keep the public game state, meaning the shot history, the turn order and the win condition, on a blockchain, and inherit its latency, its cost and its reliance on external infrastructure. This paper shows that the straightforward attempt to drop the ledger is unsound. When the party producing a proof also supplies the shot history against which “sunk” and “you win” are evaluated, a losing player can misreport sunk ships and defer a provable loss for as long as it likes while still producing proofs that verify. The correction is to bind the public shot history into the public output of every shot proof. The verifier fired exactly the shots in that history and can compare it against its own record, so in a two-player turn-based game no consensus layer is needed to fix the canonical public state. Sunk detection and the win condition are then computed inside the proof from the bound history, which makes a whole match, including the moment it ends, independently checkable from its transcript. We implement the protocol on the RISC Zero zkVM as an open-source peer-to-peer system in which each player runs a local prover and browser interface and the two are joined by an untrusted message relay. The first move is settled by a Blum coin flip, mock proving is removed at compile time, and the in-circuit game logic runs with a control flow that does not depend on the secret board. On a commodity laptop CPU a shot proof takes between 30 and 47 s to generate and about 30 ms to verify, which is fast enough for turn-based play without special hardware.

1 Introduction

Battleship is a game played on trust. Each player secretly arranges a fleet on a 10×10 grid, the opponent calls out coordinates, and the player holding the board reports each shot as a hit or a miss. Since only that player can see the board, nothing stops them from moving a ship out of the way during the game or refusing to admit that a ship has sunk. This is acceptable among friends but offers no protection against an opponent who is willing to cheat.

Cryptography removes the need to trust the opponent. Before play begins a player can *commit* to a board, and can then answer each shot with a *zero-knowledge proof* that the answer is consistent with the committed board while disclosing nothing else about it. The commit-and-prove approach is by now standard, and Battleship has been built on it several times: with hand-written arithmetic circuits checked by an Ethereum contract [12, 13], with a zkVM that posts proofs to the NEAR blockchain [14], and as sample applications on privacy-oriented chains [15, 16]. The same idea of hiding state behind commitments and proving that actions respect it underlies the on-chain game Dark Forest [17].

These systems share one design choice. The public part of the game state, which cells have been fired on, whose turn it is, how many hits have landed and hence who has won, is kept

on a blockchain, and a smart contract adjudicates everything that is not secret. The chain adds no secrecy, since the commitments and proofs already provide that. What it adds is an authoritative public history. This role is easy to miss, and a protocol that removes the chain without supplying a substitute fails for a reason that has nothing to do with the zero-knowledge machinery (Section 4). The predicates “this shot sank a ship” and “I have lost” both depend on the shot history, so a prover free to choose that history can make them come out however it likes. A losing player can keep producing valid proofs indefinitely and never be shown to have lost. We call this the *immortal fleet*.

This paper presents and evaluates a Battleship protocol that uses no ledger, no referee and no trusted third party, and in which the whole game, its ending included, can be verified. It rests on the following observation, which as far as we know has not been used before. In a two-party turn-based game, the public state a verifier needs in order to check a proof about the opponent’s hidden board is just the sequence of shots the verifier itself has fired, and the verifier already holds that sequence. Binding it into the public output of each proof therefore does the work that the ledger did.

Concretely, each shot proof takes as input the full public history of shots fired at the responding player, recomputes the commitment from the private board so that the player is tied to one board for the whole game, evaluates the hit, sunk and defeated predicates from the board together with the history, and writes the history into the proof’s public output. The verifier accepts the proof only when the history it contains matches, in order and element by element, the shots the verifier actually fired. Every receipt is then self-contained: given the two commitment receipts and the ordered shot receipts, an outside party can re-verify a complete match and determine the winner with nothing else to go on, because the history fields link the receipts to one another (Section 5.3).

Contributions.

1. We describe the *public-history binding problem* for ledger-free commit-and-prove games and give explicit attacks against the naive chain-free protocol, covering forged sunk reports and unbounded postponement of defeat (Section 4).
2. We give a complete two-party protocol, consisting of a board commitment with an in-proof legality check, a Blum commit-reveal coin flip [4] for the first move, history-bound shot proofs and an in-proof win condition, and we analyse its security properties (Sections 5–6).
3. We implement the protocol on the RISC Zero zkVM [7] with a deliberately small trust footprint: each player runs a local prover and browser UI, an untrusted stateless relay forwards only public messages, mock proving is compiled out, and every routine that reads the secret board runs a fixed-shape instruction sequence so that proving time reveals nothing about the board (Section 7). The code is open source.
4. We report measurements on a commodity laptop CPU: proving and verification times, receipt sizes for both composite and succinct receipts, the dependence on history length, and an empirical test of witness-independence (Section 8).

We are explicit about what is not new here. The commit-and-prove structure for Battleship predates this work. The contribution is to remove the ledger while preserving the integrity guarantees and, through proven termination, slightly strengthening them, and to provide a complete implementation with measurements. Zero-knowledge proofs cannot compel a player to keep playing, so aborts remain possible exactly as in the earlier systems; Section 9 treats this boundary.

2 Background

Commitments. A commitment scheme lets a party fix a value v by publishing $c = \text{Com}(v; r)$ that reveals nothing about v (*hiding*) and cannot later be opened to any $v' \neq v$ (*binding*). We use the standard salted-hash commitment $C = \text{SHA-256}(\text{encode}(B) \parallel s)$ over a canonical board encoding and a fresh 256-bit salt s . Binding rests on collision resistance and hiding on the unpredictability of the salted preimage, which is the usual random-oracle-style assumption for this construction.

Zero-knowledge proofs and zkVMs. A zero-knowledge proof [1] convinces a verifier that a statement holds without revealing anything beyond its truth, and modern succinct non-interactive systems [6, 5] make this practical for real applications. A *zkVM* lifts proving from circuits to ordinary programs: the prover runs a program on private inputs and produces a proof that it executed correctly. RISC Zero [7] proves the execution of RISC-V binaries using a STARK [5]. A proving run produces a *receipt* made of a *journal*, which holds the public outputs the program chose to commit, and a *seal*, which is the proof itself. Verification checks the seal against an *image ID*, a digest of the exact guest binary, so a verifier learns that this particular program ran and produced this particular journal, and learns nothing further. SP1 [8] and Jolt [9] are comparable systems; [10, 11] survey and benchmark the landscape.

Battleship. Two players each place one ship of every length in $\{5, 4, 3, 3, 2\}$, seventeen cells in all, on a 10×10 grid. Ships are straight, axis-aligned, within bounds and non-overlapping, and touching is permitted. Players take turns naming a cell of the opponent’s grid and learn whether it is a hit or a miss. A ship is announced sunk once all of its cells have been hit, and a player who has had all seventeen ship cells hit loses. We add the common rule that a cell may be fired on at most once per board. This caps a game at 100 shots per side and, in our protocol, keeps the two players’ history sequences aligned.

3 Related work

Cryptographic treatments of games are older than practical zero-knowledge proofs. Mental poker [2] dealt a deck between mutually distrusting players, and the completeness theorems for secure computation [3] show that in principle any game can be played fairly. What has changed since is practicality: succinct proofs and zkVMs have made per-move proving cheap enough to use.

On-chain ZK Battleship. BattleZips [12], built on circom and Groth16, proves board legality at placement and hit-or-miss on each shot, while an Ethereum contract enforces turn order, records declared shots, counts the seventeen hits that end the game, and is the root of trust for all public state. The MIT 6.857 project [13], using circom and snarkjs on Ethereum, proves that each reported result is consistent with the hashed board but, unlike BattleZips, does not prove that the committed board is legal, so a player may commit a malformed board; the win condition is again computed by the contract. RISC Zero’s own example [14] put the logic in a zkVM but stored game state in a NEAR contract, and is now unmaintained and built against an old receipt format. Aleo and Mina host comparable demonstrations [15, 16]. In each case the chain is what pins the public history, and this paper is about what takes its place once the chain is gone.

Off-chain ZK gaming. Dark Forest [17] was the first deployed game to hide state behind commitments, though its core still runs on-chain. State-channel constructions [19, 18] carry out interaction off-chain between on-chain checkpoints, and “ZK state channels” have been proposed for turn-based games, but they still rely on a chain to settle disputes. Our setting is more restrictive, in that no chain is present at any layer, and the point we make is that for

two-player turn-based games a chain is unnecessary for integrity and is useful only as a remedy against aborts (Section 9).

Fair coin flipping. The first move is chosen with Blum’s commit-reveal protocol [4]. Its familiar weakness, that the party revealing last can abort after seeing an unfavourable result, is already covered by the fact that any zero-knowledge game protocol permits aborts.

4 The public-history binding problem

Consider the natural ledger-free protocol, which up to notation is what the systems above become once the contract is removed. Player P holds a board B and salt s and has published the commitment $C = H(\text{encode}(B)\|s)$.

- **Setup proof.** P proves knowledge of a *legal* B and s with $H(\text{encode}(B)\|s) = C$; the journal is C .
- **Naive shot proof.** On shot x , P proves: “for the B, s behind C : $\text{hit} = \text{occ}(B, x)$, and $\text{sunk} = \text{sunk}(B, x, \mathcal{H})$ ”, where $\mathcal{H}$ is the set of shots previously fired at P , supplied by P as a private input. The journal is $(C, x, \text{hit}, \text{sunk})$.

The hit bit is sound, since it is a function of B and x alone and both are bound by the journal. Nothing that depends on $\mathcal{H}$ is sound, because the verifier cannot tell which $\mathcal{H}$ the statement was evaluated against. This gives three attacks.

A1: Sunk suppression. On the shot that would complete a ship, a dishonest responder proves against a history $\mathcal{H}' \subsetneq \mathcal{H}$ that omits an earlier hit. The proof verifies and reports $\text{hit} = \text{true}$, $\text{sunk} = \perp$, so the true state of the responder’s fleet is hidden. Where sunk announcements affect strategy this is a real advantage, and it costs the attacker nothing.

A2: Phantom sunk. In the other direction, the responder adds to $\mathcal{H}'$ shots that were never fired, so that a ship is announced sunk before it should be. This can be used to feign weakness or to disrupt an opponent who is tracking sunk counts.

A3: The immortal fleet. The win condition is $\text{allsunk}(B, \mathcal{H})$, and whoever evaluates it needs $\mathcal{H}$. Without a ledger, an implementation must either believe the loser’s client when it declares defeat or have the winner’s client tally sunk reports, and A1 already undermines the latter. A losing player is therefore never *provably* beaten: it can answer every shot with a valid proof, withhold the last sunk reports, and the game simply does not end. No cryptographic assumption fails; the proofs are true statements about a history that was never agreed upon. An earlier version of our own implementation had precisely this defect, with defeat recorded as a self-reported boolean outside any proof, and that is what led us to the present analysis.

On-chain systems avoid A1 through A3 because the contract is the history: shots are transactions and hits are counted in consensus. The problem is to find what plays that role when no chain is present.

The observation. In a two-player turn-based game the history $\mathcal{H}$ that matters for proofs about P ’s board is exactly the sequence of shots fired by the opponent V , and V is the party verifying those proofs and has chosen and recorded every one of them. The canonical public state already sits with the party that needs it, so it calls for binding rather than consensus. The fix follows: move $\mathcal{H}$ from the witness into the statement by writing it into the journal, and have V check it against its own record.

5 The protocol

We now describe the protocol as implemented. H is SHA-256, and $\text{encode}(B)$ lists the five ships sorted by a canonical ship identifier, each as (id, row, col, orientation) in 4 bytes, followed by the 32-byte salt, so that the commitment does not depend on the order in which placements happen to be listed.

5.1 Setup

Each player $P \in \{A, B\}$:

1. chooses a board B_P (interactively, in the UI) and samples $s_P \leftarrow \{0, 1\}^{256}$ from the OS CSPRNG;
2. runs the **commit** guest, which *asserts* $\text{legal}(B_P)$ (composition, bounds, non-overlap) and outputs journal $C_P = H(\text{encode}(B_P) \| s_P)$, yielding receipt R_P^{com} ;
3. samples a coin nonce $n_P \leftarrow \{0, 1\}^{256}$ and sends $(C_P, R_P^{\text{com}}, H(n_P))$ to the opponent;
4. on receiving the opponent's tuple, verifies R^{com} against the pinned **commit** image ID, checks that the journal equals the claimed C , and rejects a mirrored commitment $C = C_P$ (a copied commitment can never be answered, since the copier does not hold the witness, so a game against it could only stall);
5. reveals n_P ; verifies the opponent's reveal against $H(n)$; the first mover is decided by the XOR parity of the two nonces [4].

An illegal board therefore never enters a game, because no receipt exists for one. This is in contrast to [13], where placement legality is not proved.

5.2 Play

Let $F_V = (x_1, \dots, x_k)$ be the ordered shots V has fired at P so far (the *fired list*), maintained by V 's own agent. Shots must be pairwise distinct, and P 's agent refuses to answer a repeated cell.

On V 's turn, V sends a shot x . P runs the **shot** guest on private input (B_P, s_P) and public input $(C_P, x, \mathcal{H})$, where $\mathcal{H} = (x_1, \dots, x_k)$ is P 's record of shots received. The guest:

1. recomputes $H(\text{encode}(B_P) \| s_P)$ and *asserts* it equals C_P (board binding)
2. computes $\text{hit} = \text{occ}(B_P, x)$; $\text{sunk} = \text{sunkship}(B_P, x, \mathcal{H})$; $\text{def} = \text{allsunk}(B_P, \mathcal{H} \cup \{x\})$
3. commits journal $J = (C_P, x, \text{hit}, \text{sunk}, \text{def}, \mathcal{H})$ (history binding)

and returns the receipt. V accepts iff:

$$\text{Verify}(R, \text{id}_{\text{shot}}) \wedge J.C = C_P \wedge J.x = x \wedge J.\mathcal{H} = F_V,$$

where the last conjunct is *sequence* equality against V 's own fired list, and V appends x to F_V only after accepting. When $J.\text{def} = \text{true}$, V has won and holds a proof of the win that it can show to anyone; P 's own agent reads the same bit from the same journal, so the defeat is at once undeniable and impossible to fake. The roles then swap.

Since the responder's history record and the verifier's fired list each grow by exactly the shot that was accepted, they remain equal by induction. Any divergence, whether a dropped, injected, reordered or replayed message, and whether caused by the network, the relay or a cheating peer, causes the next verification to fail.

5.3 Public auditability of transcripts

A match transcript consists of the two commitment receipts together with, for each direction of play, the ordered sequence of shot receipts. An uninvolved third party can check, using only the pinned image IDs, that (i) every receipt verifies, (ii) within a direction the journal history of

receipt k equals the shots of receipts 1 through $k - 1$, so the receipts chain to one another with no need for signatures or timestamps, and (iii) the final receipt of the losing direction carries $def = \text{true}$. The end of the game is thus a property of the transcript rather than an assertion by a player. This is the ledger-free counterpart of a contract declaring the game over, and it is exactly what the naive protocol cannot provide.

5.4 Cost

Each shot costs one proof, one verification, and a journal holding $O(|\mathcal{H}|)$ shot entries at two bytes each before serialization. Across a full 100-shot direction the journal overhead summed over the transcript is $O(n^2)$, on the order of 10^4 bytes, which is small next to the seals themselves (Section 8). A running-hash binding ($h_k = H(h_{k-1} \| x_k)$) would reduce each journal to $O(1)$ but force an auditor to recompute the chain; for $n \leq 100$ the explicit list is simpler and self-describing.

6 Security analysis

The adversary is a computationally bounded player who may deviate however it wishes: it can send any messages, commit to any board, present any history, and abort at any time. The relay is trusted only to deliver messages; it can observe and drop them but never carries anything other than public data. We assume the collision resistance of SHA-256, the soundness and zero-knowledge of the RISC Zero STARK [7, 5], and the correctness of the roughly 300-line shared rules crate, which is compiled into the guest and constitutes the protocol’s entire trusted rulebook; both players pin its image IDs and check them on every verification.

Claim 1 (Response soundness). *If V accepts journal J for shot x , then except with negligible probability there exist B, s with $H(\text{encode}(B) \| s) = C_P$, $\text{legal}(B)$, and $J.\text{hit}, J.\text{sunk}, J.\text{def}$ equal to the rule functions evaluated at (B, x, F_V) .*

Sketch. STARK soundness gives an execution of the pinned guest on some witness (B, s) that produces J . The guest’s first assertion forces $H(\text{encode}(B) \| s) = J.C = C_P$, and legality of B follows from the setup receipt for C_P together with collision resistance, since two distinct boards behind one C would be a collision. The guest evaluates the rule functions on $(B, x, J.\mathcal{H})$ by construction, and V ’s check $J.\mathcal{H} = F_V$ replaces the journal’s history with the true one. $\square$

Claim 2 (Board binding across the match). *All accepted journals for commitment C_P are consistent with a single board B , except with negligible probability.*

Sketch. Each accepted journal is backed by a witness that hashes to C_P , and two witnesses carrying different boards would form a SHA-256 collision, since the encoding is injective on legal boards: it records each ship’s kind, anchor and orientation in a canonical order. $\square$

Claim 3 (Termination soundness and undeniability). *V obtains a transferable proof of victory exactly when $F_V \cup \{x\}$ covers all 17 ship cells of P ’s committed board; P cannot postpone this by any choice of messages other than refusing to answer.*

Sketch. def is computed inside the guest from $(B, x, \mathcal{H})$ with $\mathcal{H}$ bound to F_V , so by response soundness it matches the true predicate. To avoid a journal with $def = \text{true}$, a responder would have to present a different history, which V ’s comparison rejects, a different board, which is a collision, or no proof at all, which is an abort and is visible as one. Attacks A1 through A3 are ruled out in the same way. $\square$

Claim 4 (Secrecy). *A (possibly malicious) verifier interacting with an honest prover learns nothing about the prover’s board beyond the game-inherent information: the hit/miss/sunk/defeated values of the queried cells, and what those imply.*

Sketch. The prover’s messages are receipts, and by the zero-knowledge property of the proof system the verifier’s view can be simulated from the journals alone. Those journals contain only C , which hides the board through the 256-bit salt, the verifier’s own shots, and the game-inherent result bits. The guarantee is deliberately one of a leaky function: the answers in Battleship themselves shrink the space of possible boards, which is the game working as intended, and the protocol protects only what remains. $\square$

Replay, reflection, reordering. Journals bind $(C, x, \mathcal{H})$, so a replayed receipt fails on the x or $\mathcal{H}$ check, and a receipt from the opposite direction of play refers to the wrong commitment. An opponent that mirrors a commitment is rejected during setup and could not answer shots in any case.

Malicious relay. The relay carries commitments, coin values, shot coordinates and receipts, all of which are public. It cannot change any of them without detection, since verification fails on a mismatch; it cannot forge outcomes, since it cannot produce receipts for the pinned image IDs; and it learns nothing that the transcript does not already contain. Its only real power is to deny service.

Prover-side side channels. Proof contents reveal nothing beyond the journals, but the time taken to prove is visible to the opponent. If the guest’s control flow depended on ship positions, this latency could leak information about the board. Every routine in the shared crate that reads the board is therefore written with fixed-shape loops, using the fact that a legal fleet always occupies exactly seventeen cells, and with boolean accumulation that does not short-circuit, so that the length of the instruction trace does not depend on the witness. Section 8 checks this invariance empirically. We make no claim about physical side channels on the prover’s own machine, which lie outside our scope.

7 Implementation

The system comprises roughly 3,400 lines of Rust and TypeScript and is open source under the MIT license.

- **core:** the shared rules crate discussed above, defining the types, fleet legality, the hit, sunk and defeat predicates, and the canonical encoding. It is compiled unchanged into both the native host and the RISC-V guest, so the rules that are proved are identical to the rules the application plays. Its unit tests include an equivalence suite that checks the fixed-shape implementations against straightforward reference versions over random boards.
- **methods/guest:** the two guest programs, **commit** and **shot**, each about 40 lines on top of **core**. SHA-256 uses the zkVM’s accelerated implementation.
- **host:** the prover and verifier library together with the *prover agent*, a localhost-only HTTP service that holds the player’s board and salt, proves the player’s answers, and verifies the opponent’s, checking the receipt, image ID, commitment, shot and history. The build turns on RISC Zero’s `disable-dev-mode`, and this together with clearing `RISCO_DEV_MODE` from the environment makes mock receipts impossible rather than merely discouraged. Adversarial integration tests run each attack from Section 4 against the real prover and check that it fails.
- **relay:** a WebSocket forwarder of about 200 lines, with no game logic and no dependency on RISC Zero, deployable as a container anywhere. It pairs two connections by room code and passes messages between them.

- **web**: a Next.js interface for each player, covering fleet placement, firing and status, which communicates only with that player’s own agent and with the relay. The Blum coin flip runs here through WebCrypto, within the trust domain of the player it protects.

Two properties of the deployment are worth stating. First, a secret is confined to a single process: the board and salt exist only inside the player’s own agent, which binds to 127.0.0.1 and answers CORS requests only from the local UI origin. Second, each agent exposes its guest image IDs at `/health`, so players can compare them out of band, although a mismatch is in any event unexploitable and merely causes every cross-verification to fail.

8 Evaluation

Setup. All measurements are CPU-only on a consumer laptop: Intel Core Ultra 7 155H (22 hardware threads), 15 GB RAM available to the measurement environment (WSL2, Ubuntu 22.04), `rustc` 1.96, `risc0-zkvm` 3.0.5, default composite receipts and `ProverOpts::succinct()` where indicated. Wire size is the JSON-serialized receipt as actually sent through the relay.

Proof	Receipt	$ \mathcal{H} $	Prove (s)	Verify (ms)	Journal (B)	Seal (KiB)	Wire (KiB)
commit	composite	–	21.0	34.5	128	215.7	554.1
shot	composite	0	30.0	34.3	152	215.7	554.1
shot	composite	25	47.2	27.1	352	238.2	613.6
shot	composite	50	41.1	28.2	552	238.2	614.4
shot	composite	99	42.3	32.8	944	238.2	616.1
commit	succinct	–	54.3	31.1	128	217.4	569.5
shot	succinct	0	54.8	29.0	152	217.4	569.5
shot	succinct	99	76.8	27.1	944	217.4	572.8

Table 1: Proving and verification cost (single runs; proving uses all 22 threads). $|\mathcal{H}|$ is the length of the bound shot history. Wire size is the JSON-serialized receipt as sent through the relay; Seal is the raw STARK seal within it.

Latency and playability. Setup requires one commit proof per player, about 21 s and produced concurrently on the two machines, plus a coin flip that takes under a millisecond. Each subsequent turn requires one shot proof, between 30 and 47 s, along with verification of about 30 ms and the relay round-trips. Guest execution grows linearly in the bound history, from 20.3k user cycles at $|\mathcal{H}| = 0$ to 101.8k at $|\mathcal{H}| = 99$, but the prover pads execution into segments of a power-of-two length, so the cost of proving changes only when a segment boundary is crossed: an empty history occupies one segment of 2^{16} cycles, and every longer history in a full game fits within two. Binding the history therefore adds almost nothing to the proving time, and its journal overhead stays under 1 KB against a receipt of about 600 KiB even at $|\mathcal{H}| = 99$, so the $O(n^2)$ transcript overhead noted in Section 5 does not matter in practice. A complete match of roughly 60 to 90 shots spends between 30 and 60 minutes proving, spread across the game at the pace of a correspondence match, and no single wait exceeds a minute on this hardware. More cores, a GPU, or a newer prover [11] would reduce these times.

Succinct receipts compress an entire execution into a seal of constant size, 217.4 KiB regardless of the history, at between 1.4 and 1.8 times the proving cost. At this circuit size a composite receipt spans only one or two segments, so succinctness saves nothing on the wire, and we use composite receipts by default. Recursion becomes worthwhile only when many moves are to be aggregated (Section 9).

Witness independence. Running the shot guest on a fixed shot and a 30-shot history across 20 random boards gives execution lengths between 45,147 and 45,173 cycles, a spread of 26 cycles, below 0.06%, attributable to runtime bookkeeping outside the rule functions. The spread disappears once proving begins, since all 20 executions pad to the same 2^{16} -cycle segment and therefore take the same time to prove, leaving no signal about the board. With the fixed-shape rule implementations, this gives empirical support for the side-channel claim of Section 6 rather than resting on inspection of the code alone.

Comparison with on-chain designs. A direct quantitative comparison with on-chain designs would not be meaningful, given the different trust models and hardware, but the qualitative difference is clear. On-chain systems pay consensus latency, on the order of tens of seconds per move on proof-of-work Ethereum in [13], together with a transaction fee per move, and they place game metadata in public permanently. Our design pays only local proving time and the relay round-trips, keeps the transcript private to the two players, who may publish it if they want it audited, and forgoes the chain’s built-in penalty for aborting, namely stake slashing, which we turn to next.

9 Limitations and scope

Aborts. No zero-knowledge protocol can compel a losing player to keep generating proofs, and a player facing defeat can simply disconnect. What the protocol does guarantee is that such an abort is attributable and cannot be faked, since the transcript shows who stopped answering and that every answered shot was honest, and that the outcome of a completed game is not open to dispute. Remedies such as forfeit timers by social convention, or stakes with slashing for anyone who chooses to add a chain or an escrow, sit outside the protocol and compose with it unchanged.

Trusted rulebook. Soundness is relative to the correctness of the roughly 300-line rules crate and of the zkVM toolchain. The crate is small, is shared verbatim between play and proof, and is tested adversarially; trust in the toolchain is common to the whole zkVM ecosystem [10].

Metadata. The protocol conceals boards rather than behaviour. Timing, choice of moves and disconnections are visible to the opponent and the relay, and proving-time side channels are reduced by the fixed-shape traces but not physically eliminated. Reusing a board or a salt across games invalidates the hiding argument, and the implementation draws a fresh salt for each game and advises against reusing boards.

Generality. The construction extends directly to any two-player turn-based game with hidden state whose public state is a deterministic function of the public sequence of moves, such as Stratego-style reveals or fog-of-war variants, by substituting the rule functions inside the guest. Games with hidden moves, simultaneous turns, or more than two players bring back the need for a shared ordering or a fairness layer and are outside our scope.

10 Conclusion

Deployed zero-knowledge Battleship systems hand the public half of the game, its history, turns and termination, to a blockchain. We have shown that removing the chain without more care breaks exactly that half, producing unverifiable sunk reports and the immortal fleet, and that for two-player turn-based games no consensus is needed to repair it. The verifier already holds the canonical history, and binding it into each proof’s public output, together with computing the

win condition inside the proof, yields self-contained receipts, matches that terminate verifiably, and a deployment that consists of nothing more than the two players and a message relay. The implementation runs on laptop CPUs and is open source, and we intend it as a compact and complete example of a trustless hidden-state game built without a chain.

Artifact

Source code (protocol, agent, relay, UI, benchmarks, and the LaTeX for this paper) is available at <https://github.com/hadigghazi/Ledger-Free-Trustless-Battleship>, and the v1.0.0 release is archived at DOI 10.5281/zenodo.21207531. All reported numbers are reproducible via `cargo run -release -p host -bin zk-battleship-bench` and the `-ignored` integration test suite.

References

- [1] S. Goldwasser, S. Micali, and C. Rackoff. The knowledge complexity of interactive proof systems. *SIAM Journal on Computing*, 18(1):186–208, 1989.
- [2] A. Shamir, R. L. Rivest, and L. M. Adleman. Mental poker. In *The Mathematical Gardner*, pages 37–43. Springer, 1981.
- [3] O. Goldreich, S. Micali, and A. Wigderson. How to play any mental game, or a completeness theorem for protocols with honest majority. In *Proc. 19th ACM STOC*, pages 218–229, 1987.
- [4] M. Blum. Coin flipping by telephone: a protocol for solving impossible problems. *ACM SIGACT News*, 15(1):23–27, 1983.
- [5] E. Ben-Sasson, I. Bentov, Y. Horesh, and M. Riabzev. Scalable, transparent, and post-quantum secure computational integrity. Cryptology ePrint Archive, Report 2018/046, 2018.
- [6] J. Groth. On the size of pairing-based non-interactive arguments. In *EUROCRYPT 2016*, LNCS 9666, pages 305–326. Springer, 2016.
- [7] J. Bruestle and P. Gafni. RISC Zero zkVM: scalable, transparent arguments of RISC-V integrity. RISC Zero, 2023. <https://dev.risczero.com/proof-system-in-detail.pdf>.
- [8] Succinct Labs. SP1: a performant, open-source zkVM. 2024. <https://github.com/succinctlabs/sp1>.
- [9] A. Arun, S. Setty, and J. Thaler. Jolt: SNARKs for virtual machines via lookups. In *EUROCRYPT 2024*, LNCS 14656, pages 3–33. Springer, 2024.
- [10] R. Lavin, X. Liu, H. Mohanty, L. Norman, G. Zaarour, and B. Krishnamachari. A survey on the applications of zero-knowledge proofs. arXiv:2408.00243, 2024.
- [11] T. Gassmann, S. Chaliasos, T. Sotiropoulos, and Z. Su. Evaluating compiler optimization impacts on zkVM performance. In *Proc. ASPLOS 2026*. ACM, 2026. arXiv:2508.17518.
- [12] BattleZips. BattleZips-Circom: zero-knowledge arithmetic circuits for on-chain Battleship. 2022. <https://github.com/BattleZips/BattleZips-Circom>.
- [13] A. Gupta, N. Kaashoek, B. Wang, and J. Zhao. Zero-knowledge Battleship. MIT 6.857 course project report, 2020. <https://courses.csail.mit.edu/6.857/2020/projects/13-Gupta-Kaashoek-Wang-Zhao.pdf>.
- [14] RISC Zero. Battleship example on NEAR. 2022. <https://github.com/risc0/battleship-example>.
- [15] Aleo. ZK Battleship in Leo (workshop example). 2023. <https://github.com/AleoNet/workshop>.

- [16] Mina Foundation. Battleships: a zero-knowledge game on Mina with o1js. 2023. <https://github.com/o1-labs>.
- [17] Dark Forest team. Announcing Dark Forest. 2020. <https://blog.zkga.me/announcing-darkforest>.
- [18] A. Kosba, A. Miller, E. Shi, Z. Wen, and C. Papamanthou. Hawk: the blockchain model of cryptography and privacy-preserving smart contracts. In *IEEE S&P 2016*, pages 839–858, 2016.
- [19] J. Coleman, L. Horne, and L. Xuanji. Counterfactual: generalized state channels. 2018. <https://14.ventures/papers/statechannels.pdf>.